Qué estamos haciendo

Una plataforma ficticia de reparto para demostrar una base de datos analítica.

Guardamos eventos. Mostramos lo que ocurre.

ClickHouse conserva los registros. El dashboard muestra pedidos, acciones de navegación y repartidores simulados que se mueven.

Dos tipos de datos, dos objetivos

Fuentes públicas

OTTO + REES46: cientos de millones de eventos históricos.

Ele.me: muestras de recomendaciones.
Glovo: productos de catálogo.

Nos ayudan a estudiar volumen y consultas.

Nuestro escenario

Madrid: 12 repartidores simulados por defecto.

Pedidos, importes y posiciones ficticios.
Acciones generadas o manuales.

Nos ayuda a demostrar nuestra infraestructura en funcionamiento.

ClickHouse no captura clics solo: nuestro código crea o recibe registros y la base permite consultarlos. El mapa lee posiciones que han pasado por la cola y la base.

Estado del proyecto

Ya tenemos: base, ingesta, simulación, dashboard y web compartida.
Falta: miniapp comercial con checkout conectado a pedidos y reproducción comprobada en otro ordenador.

Fuentes y publicación comprobadas el 7/10. No mezclamos sus IDs ni afirmamos que hoy esté encendido el laboratorio.

Cómo se conecta todo

Las flechas muestran el recorrido de los datos nuevos hasta el panel.



El dashboard pide datos a la API; la API consulta ClickHouse. Validación y lectura son parte del mismo programa. El simulador integrado usa directamente su validación y cola.

Docker conecta los servicios

Tenemos un solo nodo ClickHouse. La API y la cola no son otros nodos de esa base.

Qué guardamos y qué vemos

Un clic, un estado de pedido y una posición son registros diferentes.

app_events

Una acción de navegación
Buscar, abrir un producto o añadir al carrito. El panel cuenta estas acciones.

order_events

Un pedido en un estado
Creado, en reparto, entregado o cancelado. El importe es ficticio.

courier_positions

Una posición de un repartidor
ID, hora y coordenadas. El panel dibuja el marcador y el recorrido.

Ejemplo: una posición nueva

Se mueve → **Espera en cola** → **Llega a la base** → **El mapa la lee**

Aceptar en cola no significa que ya se vea. Reenviar el mismo ID y contenido no añade otra acción lógica. Una posición antigua que llega tarde no sustituye a una más reciente.

La frecuencia no garantiza la demora

Solicitamos posiciones cada 2 s. El panel espera 2 s después de terminar su consulta. Puede haber retraso: mostramos la antigüedad de la posición.

Históricos: delivery. Plataforma propia: delivery_atlas. Son dos bases lógicas en el mismo servidor; sus poblaciones no se mezclan.

Cómo empezar juntos

Podéis probar la demo pequeña sin descargar los datasets masivos.

1. Ver el panel compartido

delivery-atlas-demo.vercel.app

Solo lectura. Se actualiza mientras el ordenador de Jere, Docker y el túnel estén activos.

2. Probarlo en vuestro ordenador

Obtener la rama codex/actualizacion-clickstream-eleme mientras la PR no esté integrada. Con Docker/Compose, desde la carpeta del proyecto:

```
docker compose -f compose.yaml --profile atlas up -d --build --wait
```

Abrir http://127.0.0.1:8001/, iniciar la simulación, elegir un repartidor y mirar cómo cambian su hora y coordenadas.

3. Por dónde empezar cada uno

Anuar

Entender cómo se registra una acción.

Mirar: atlas/contract.py y el botón manual del panel.
Siguiente paso posible: miniapp comercial.

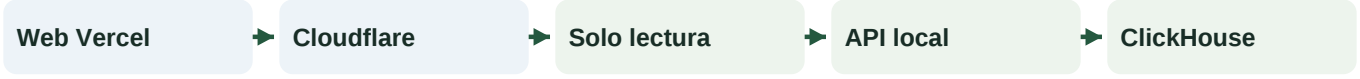
Echenique

Seguir las consultas y probar el panel local.

Mirar: atlas/clickhouse.py y web/atlas-dashboard.js.
Primer objetivo: reproducir la demo.

Jere + Rayo apoyan base, ingesta y simulación. Son rutas de incorporación; el reparto definitivo se acuerda juntos.

Así llega una consulta desde Vercel



ClickHouse y el simulador siguen locales. Los controles de iniciar/parar están en la demo local. Detalle completo: documento técnico de 17 páginas y README del repositorio.